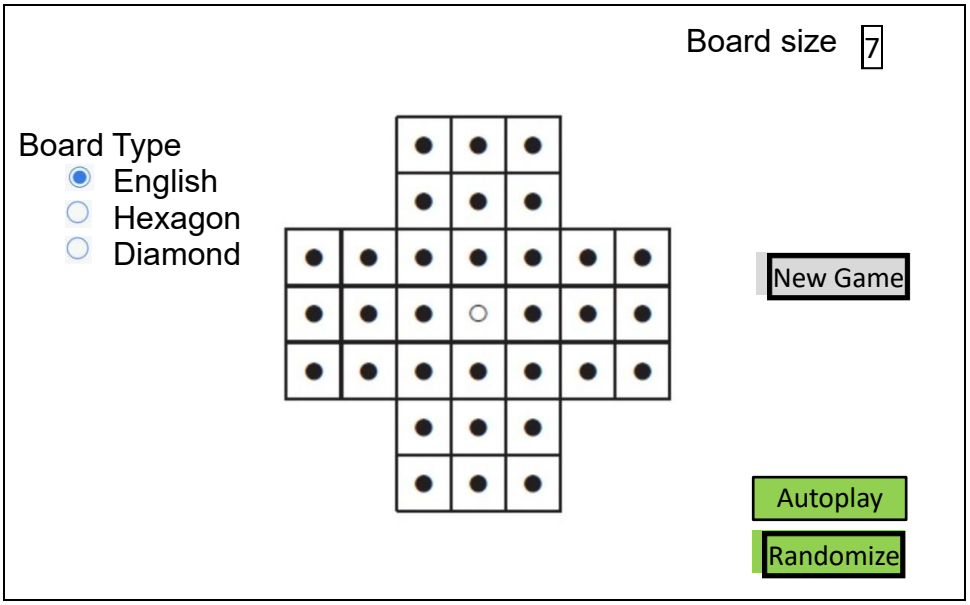
Project Sprint #3

GitHub link: https://github.com/hafsaahmed05/CS449_Solitaire

Implement all the features that support a human player to play a manual or automated Solitaire game against a human opponent and refactor your existing code if necessary. The minimum features include **choosing the game mode (manual or automated), choosing the board size, setting up a new game, making a move (in a manual or automated game), randomize the state of the board** and **determining if a manual or automated game is over**. The following is a sample GUI layout. It is required to use a class hierarchy to deal with the common requirements of the Manual Game and the Automated Game. **If your code for Sprint 2 has not considered class hierarchy, it is time to refactor your code**.



Deliverables: expand and improve your submission for sprint 2.

1. **Demonstration (9 points)**
   Submit a video of no more than five minutes, clearly demonstrating the following features.
   (a) A manual game until the game is over for English board
   (b) A manual game until the game is over for Hexagon board
   (c) An automated game until the game is over
   (d) A manual game with periodic randomization of states until the game is over
   (e) Some automated unit tests for the manual game mode
   (f) Some automated unit tests for the automated game mode

   In the video, you must explain what is being demonstrated.

2. **Summary of Source Code (1 points)**

Source code file name	Production code or test code?	# lines of code
SolitaireModel.java	Production	~216

EnglishModel.java	Production	~35
HexagonModel.java	Production	~50
DiamondModel.java	Production	~44
BoardPanel.java	Production	~126
CellButton.java	Production	~118
HomePanel.java	Production	~136
SolitaireController.java	Production	~170
SolitaireView.java	Production	~234
SolitaireModelTest.java	Test	~586
SolitaireModelTestHelper.java	Test	~250
Main.java	Production	~18
	Total	1983

3. Production Code vs User stories/Acceptance Criteria (3 points)

Summarize how each of the user story/acceptance criteria is implemented in your production code (class name and method name etc.)

User Story ID	User Story Name
1	Choose a board size and type
2	Choose the game mode of a chosen board (manual/automated)
3	Start a new game of the chosen board size and game mode
4	Make a move in a manual game
5	A manual game is over
6	Make a move in an automated game
7	An automated game is over
8	Randomize the state of the board during a manual game

User Story ID	AC ID	Class Name(s)	Method Name(s)	Status	Notes
1	1.1	EnglishModel, DiamondModel, HexagonModel	getBoardSize()	Complete	Supports multiple sizes
	1.2	SolitaireModel	getBoardCopy()	Complete	Returns correct dimensions
2	2.1	EnglishModel	initBoard()	Complete	Initializes English board
	2.2	DiamondModel	initBoard()	Complete	Diamond layout
	2.3	HexagonModel	initBoard()	Complete	Hexagon layout

3	3.1	SolitaireModel	initBoard()	Complete	Resets board
4	4.1	SolitaireModel	makeMove()	Complete	Executes legal moves
	4.2	SolitaireModel	isLegalMove()	Complete	Validates moves
5	5.1	SolitaireModel	isGameOver()	Complete	Detects no moves
	5.2	SolitaireModel	isWon()	Complete	Detects win condition
6	6.1	SolitaireModel	makeAnyMove()	Complete	Automated move
7	7.1	SolitaireModel	isGameOver()	Complete	Works with autoplay
8	8.1	SolitaireModel	randomizeBoard()	Complete	Random board state

4. Tests vs User stories/Acceptance Criteria (2 points)

Summarize how each of the user story/acceptance criteria is tested by your test code (class name and method name) or manually performed tests.

User Story ID	User Story Name
1	Choose a board size and type
2	Choose the game mode of a chosen board (manual/automated)
3	Start a new game of the chosen board size and game mode
4	Make a move in a manual game
5	A manual game is over
6	Make a move in an automated game
7	An automated game is over
8	Randomize the state of the board during a manual game

4.1 Automated tests directly corresponding to some acceptance criteria

User Story 1 – Choose board size and type

User Story ID	AC ID	Class Name	Method Name	Status
1	1.1	SolitaireModelTest	testBoardSizeIsSeven	Pass
1	1.2	SolitaireModelTest	testBoardDimensions	Pass
1	1.3	SolitaireModelTest	testBoardSizeNine	Pass

User Story ID	AC ID	Class Name	Method Name	Status
1	1.4	SolitaireModelTest	testDiamondBoardSize	Pass
1	1.5	SolitaireModelTest	testHexagonBoardSize	Pass

User Story 2 – Choose board type

English Board

User Story ID	AC ID	Class Name	Method Name	Status
2	2.1	SolitaireModelTest	testEnglishInitialPegCount	Pass
2	2.2	SolitaireModelTest	testEnglishCentreCellIsEmpty	Pass
2	2.3	SolitaireModelTest	testEnglishCornerIsInvalid	Pass
2	2.4	SolitaireModelTest	testEnglishValidCellLayout	Pass

Diamond Board

User Story ID	AC ID	Class Name	Method Name	Status
2	2.5	SolitaireModelTest	testDiamondCentreCellIsEmpty	Pass
2	2.6	SolitaireModelTest	testDiamondCornerIsInvalid	Pass
2	2.7	SolitaireModelTest	testDiamondMiddleRowFullyValid	Pass
2	2.8	SolitaireModelTest	testDiamondTopCell	Pass

Hexagon Board

User Story ID	AC ID	Class Name	Method Name	Status
2	2.9	SolitaireModelTest	testHexagonCentreCellIsEmpty	Pass
2	2.10	SolitaireModelTest	testHexagonCornerIsInvalid	Pass
2	2.11	SolitaireModelTest	testHexagonMiddleRowFullyValid	Pass
2	2.12	SolitaireModelTest	testHexagonTopRowNarrowerThanMiddle	Pass

User Story 3 – Start a new game

User Story ID	AC ID	Class Name	Method Name	Status
3	3.1	SolitaireModelTest	testNewGameResetsPegCount	Pass
3	3.2	SolitaireModelTest	testNewGameResetsCentreCell	Pass
3	3.3	SolitaireModelTest	testNewGameRestoresPeg	Pass
3	3.4	SolitaireModelTest	testHexagonInitBoard	Pass
3	3.5	SolitaireModelTest	testDiamondInitBoard	Pass

User Story 4 – Make a move in a manual game

User Story ID	AC ID	Class Name	Method Name	Status
4	4.1	SolitaireModelTest	testLegalMoveReturnsTrue	Pass
4	4.2	SolitaireModelTest	testAfterMoveSourceIsEmpty	Pass
4	4.3	SolitaireModelTest	testAfterMoveJumpedCellIsEmpty	Pass
4	4.4	SolitaireModelTest	testAfterMoveDestinationHasPeg	Pass
4	4.5	SolitaireModelTest	testLegalMoveDecrementsPegCount	Pass
4	4.6	SolitaireModelTest	testIllegalMoveReturnsFalse	Pass
4	4.7	SolitaireModelTest	testMoveToOccupiedCellReturnsFalse	Pass
4	4.8	SolitaireModelTest	testIllegalMoveDoesNotChangePegCount	Pass
4	4.9	SolitaireModelTest	testDiagonalMoveIsIllegal	Pass
4	4.10	SolitaireModelTest	testIsLegalMoveBeforeAndAfter	Pass
4	4.11	SolitaireModelTest	testHexagonLegalMove	Pass
4	4.12	SolitaireModelTest	testDiamondLegalMove	Pass

User Story 5 – A manual game is over

User Story ID	AC ID	Class Name	Method Name	Status
5	5.1	SolitaireModelTest	testNewBoardNotGameOver	Pass
5	5.2	SolitaireModelTest	testNewBoardNotWon	Pass
5	5.3	SolitaireModelTest	testOnePegIsWon	Pass
5	5.4	SolitaireModelTest	testNoMovesIsGameOver	Pass
5	5.5	SolitaireModelTest	testHexagonGameOver	Pass
5	5.6	SolitaireModelTest	testDiamondGameOver	Pass

User Story 6 – Make a move in an automated game

User Story ID	AC ID	Class Name	Method Name	Status
6	6.1	SolitaireModelTest	testMakeAnyMoveReturnsTrueWhenMoveAvailable	Pass
6	6.2	SolitaireModelTest	testMakeAnyMoveDecrementsPegs	Pass
6	6.3	SolitaireModelTest	testMakeAnyMoveReturnsFalseWhenStuck	Pass
6	6.4	SolitaireModelTest	testMakeAnyMoveBoardConsistency	Pass
6	6.5	SolitaireModelTest	testMakeAnyMoveHexagon	Pass
6	6.6	SolitaireModelTest	testMakeAnyMoveDiamond	Pass

User Story 7 – Automated game is over

User Story ID	AC ID	Class Name	Method Name	Status
7	7.1	SolitaireModelTest	testAutoplayEventuallyEndsGame	Pass
7	7.2	SolitaireModelTest	testAutoplayPegCountAtGameOver	Pass
7	7.3	SolitaireModelTest	testAutoplayHexagonEndsGame	Pass

User Story ID	AC ID	Class Name	Method Name	Status
7	7.4	SolitaireModelTest	testAutoplayDiamondEndsGame	Pass

User Story 8 – Randomize board

User Story ID	AC ID	Class Name	Method Name	Status
8	8.1	SolitaireModelTest	testRandomizeOnlyValidStates	Pass
8	8.2	SolitaireModelTest	testRandomizeCentreIsEmpty	Pass
8	8.3	SolitaireModelTest	testRandomizePegCountIsConsistent	Pass
8	8.4	SolitaireModelTest	testRandomizeChangesBoardState	Pass
8	8.5	SolitaireModelTest	testRandomizeHexagon	Pass
8	8.6	SolitaireModelTest	testRandomizeDiamond	Pass

4.2 Manual tests directly corresponding to some acceptance criteria

User Story	AC ID	Input	Expected Output	Notes
2	2.1	Select English board	English board appears correctly	GUI test
2	2.2	Select Diamond board	Diamond board appears correctly	GUI test
2	2.3	Select Hexagon board	Hexagon board appears correctly	GUI test
3	3.1	Click “New Game”	Board resets	UI verified
4	4.1	Click valid move	Peg moves correctly	Manual interaction
6	6.1	Click autoplay	Moves happen automatically	Visual confirmation
8	8.1	Click randomize	Board changes randomly while keeping valid structure	Verified visually

5. **(Part 1) Describe how the class hierarchy in your design deals with the common and different requirements of the Manual Game and the Automated Game.**
   At least 1/2 page, excluding screenshots/diagrams (12pt, single-spaced), is required. (3 Points)

The Solitaire game is designed using an object-oriented class hierarchy combined with an MVC structure. The base class SolitaireModel contains the core game logic which includes the following primary methods -- makeMove(), isLegalMove(), isGameOver(), isWon(), makeAnyMove(), and randomizeBoard(). It also manages the board state and provides helper methods for accessing it.

The subclasses EnglishModel, DiamondModel, and HexagonModel extend SolitaireModel and implement board-specific behavior. Each subclass overrides methods like initBoard() and getDirections() to define its own layout and movement rules, allowing different board types to reuse shared logic while customizing only what is necessary.
The system also follows an MVC design. The SolitaireView, along with BoardPanel, CellButton, and HomePanel, handles the graphical interface. The SolitaireController processes user input and connects the view to the model, while the Main class initializes the application.

This design supports both manual and automated gameplay within the same architecture. The manual game mode relies on user-triggered actions handled through the controller, which then calls makeMove(). The automated game mode uses the makeAnyMove() method repeatedly to simulate moves until the game ends. Because both modes use the same model, there is no duplication of logic.

Overall, this architecture demonstrates strong adherence to object-oriented design principles.

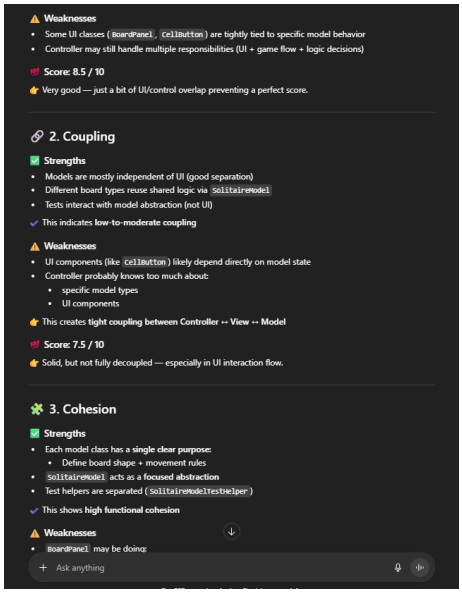
- Modularity is achieved by separating responsibilities across model, view, and controller classes.
- Cohesion is high, as each class has a clear and focused role.
- Coupling is reduced because components interact through well-defined interfaces rather than direct dependencies.
- Encapsulation is maintained by keeping the board state private within the model and exposing controlled access through public methods.

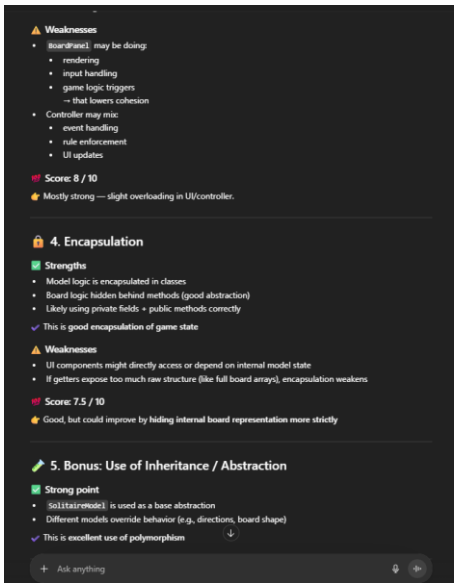
(Part 2) Demonstrate how you use LLM (ChatGPT or other) to analyze how well your code adheres to the design principles discussed in class - modularity, cohesion, coupling, and encapsulation – using the definitions provided in class. Provide screenshots of your interactions with the LLM, showing the prompts used and the responses received.

Interpret the LLM's feedback, refine your prompts if needed to obtain more relevant responses, and discuss any changes made to your code based on the analysis. Explain how these refinements improve adherence to the design principles.

At least 1/2 page is required, excluding screenshots (12pt, single-spaced). (2 Points)

To evaluate how well the code adheres to object-oriented design principles, I used both ChatGPT and Claude to analyze my implementation of the Solitaire game. I provided my GitHub link as well as referred to certain parts of my code like the SolitaireModel base class and its subclasses (EnglishModel, DiamondModel, and HexagonModel). I prompted the LLMs to evaluate the design in terms of modularity, cohesion, coupling, and encapsulation using the definitions discussed in class.





Final Scores

Principle	Score
Modularity	8.5 / 10
Coupling	7.5 / 10
Cohesion	8 / 10
Encapsulation	7.5 / 10

Final Verdict

Overall: 8 / 10 (Strong Design)

Your code:

- Clearly follows **MVC principles**
- Uses **inheritance well**
- Is **organized and readable**

What Would Push This to a 9–10?

1. Reduce Controller Responsibility

- Move more logic into model or helper classes
  - Controller should just coordinate

2. Decouple UI from Model

Instead of:

```java
button.setState(model.getBoard()[r][c]);
```

Use:

```java
model.getCellState(r, c);
```

Prevents UI from knowing internal structure

3. Improve Encapsulation

- Avoid exposing full board arrays
- Provide **intent-based methods** instead:
  - `isValidMove(...)`
  - `getCellState(...)`

4. Improve Cohesion in UI Classes

Split responsibilities:

- Rendering → `BoardPanel`
- Input handling → Controller

If You Want (Highly Recommended for Your Assignment)

Ask anything

ChatGPT can make mistakes. Check important info.

The analysis produced by ChatGPT shows that my project follows a very clear MVC-style architecture. My project demonstrated a strong use of modularity with the separation between the model (SolitaireModel and its subclasses), the view (SolitaireView, BoardPanel, CellButton, HomePanel), and the controller (SolitaireController). It also noted good cohesion, as each class generally has a focused responsibility, such as handling game logic, UI rendering, or user interaction.

However, the LLM analysis also pointed out several areas for improvement. It suggested that some UI components, such as BoardPanel and CellButton, are somewhat tightly coupled with the model, and that the controller may be handling too many responsibilities, including both UI coordination and aspects of game flow. Additionally, it identified that encapsulation could be improved by avoiding direct exposure of the internal board representation and instead consistently using accessor methods like getCellState().

At this stage, I have not yet implemented these suggested changes, but I plan to incorporate them in the next sprint. These improvements would strengthen encapsulation by protecting internal state, reduce coupling between UI and model components, and improve cohesion by ensuring that each class maintains a single, well-defined responsibility.

Overall, using LLMs such as ChatGPT helped me critically evaluate my design and identify clear areas for improvement, even before making code changes. It gave me a lot of useful guidance for refining the architecture as this project will continue to develop.